

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN NĂM
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Sau khi đã hoàn thành việc huấn luyện mô hình YOLOv8s và xây dựng API dự đoán cơ bản trong tuần thứ tư, tuần thứ năm của dự án tập trung vào việc nâng cấp hệ thống từ mức demo lên một hệ thống có tính ứng dụng thực tế cao hơn.

Các mục tiêu chính trong tuần này bao gồm:

- Triển khai chức năng giám sát video theo thời gian thực
- Tích hợp cơ sở dữ liệu để lưu trữ lịch sử vi phạm
- Giải quyết bài toán lưu trữ ảnh bằng dịch vụ cloud
- Tối ưu hóa kiến trúc hệ thống theo hướng module hóa và dễ mở rộng
- Cải thiện hiệu năng và trải nghiệm hệ thống thông qua xử lý bất đồng bộ
- Xây dựng cơ chế xử lý lỗi tập trung
- Việc hoàn thành các mục tiêu này giúp hệ thống tiến từ giai đoạn “API demo” sang “hệ thống backend có khả năng triển khai thực tế”.

2. Phát triển hệ thống giám sát và lưu trữ đám mây

Trong tuần này, hệ thống đã được nâng cấp toàn diện để đáp ứng các yêu cầu thực tế của một hệ thống giám sát thông minh.

2.1. Triển khai luồng livestream video

Để ứng dụng có tính thực tiễn cao, thay vì chỉ dự đoán ảnh tĩnh, em đã xây dựng tính năng giám sát trực tiếp từ Camera.

Cách tiếp cận:

- Sử dụng StreamingResponse của FastAPI
- Áp dụng giao thức MJPEG streaming (multipart/x-mixed-replace)
- Đọc video theo từng frame từ camera (OpenCV)
- Áp dụng mô hình đã train để dự đoán trên từng frame
- Vẽ bounding box và trả stream liên tục về client

Quy trình xử lý:

Camera => Frame => YOLO => Draw bbox => Stream về client

Để đảm bảo hiệu năng:

- Giữ kích thước ảnh đầu vào `imgsz=416`
- Giảm tải bằng cách thêm delay nhỏ giữa các frame (`asyncio.sleep`)

- Tránh xử lý quá nhanh gây quá tải GPU

Kết quả đạt được:

- Hệ thống đạt tốc độ khoảng 25-30 FPS
- Đảm bảo video hiển thị mượt, phù hợp với yêu cầu real-time

2.2. Tích hợp MongoDB Atlas và Cloudinary

Hệ thống hiện tại không chỉ "nhìn" mà còn có khả năng "ghi nhớ" lâu dài thông qua kiến trúc Hybrid-Cloud.

2.2.1. MongoDB Atlas (lưu metadata)

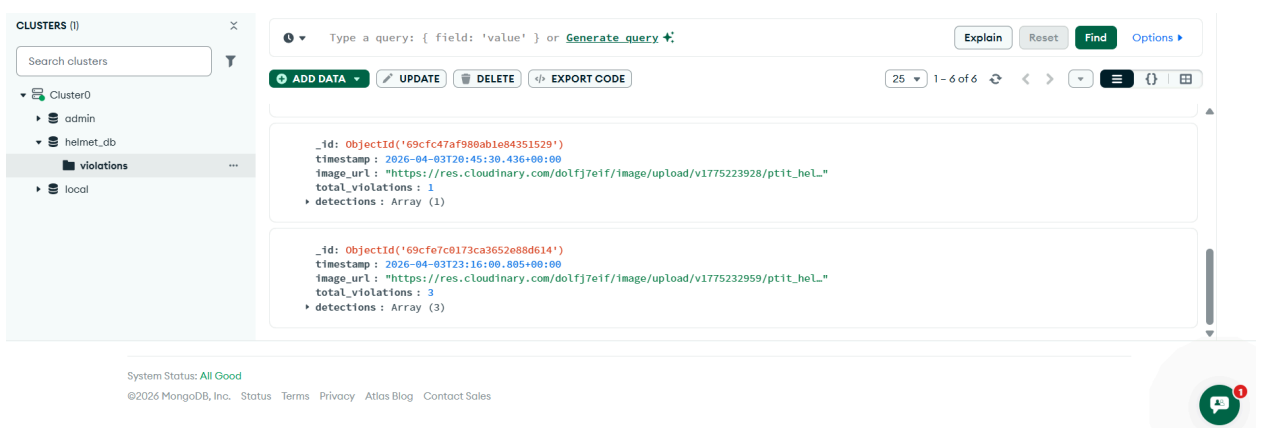
MongoDB được sử dụng để lưu trữ thông tin về các lần phát hiện vi phạm, bao gồm:

- Thời gian phát hiện
- Loại đối tượng (With Helmet / Without Helmet)
- Tọa độ bounding box
- Đường dẫn ảnh

Lý do lựa chọn:

- NoSQL linh hoạt, phù hợp với dữ liệu dạng JSON
- Dễ lưu trữ danh sách nhiều object detection trong một record
- Dễ mở rộng khi hệ thống lớn lên

Ảnh minh họa:



2.2.2. Cloudinary (lưu ảnh)

Thay vì lưu ảnh trực tiếp trên server, hệ thống sử dụng Cloudinary để:

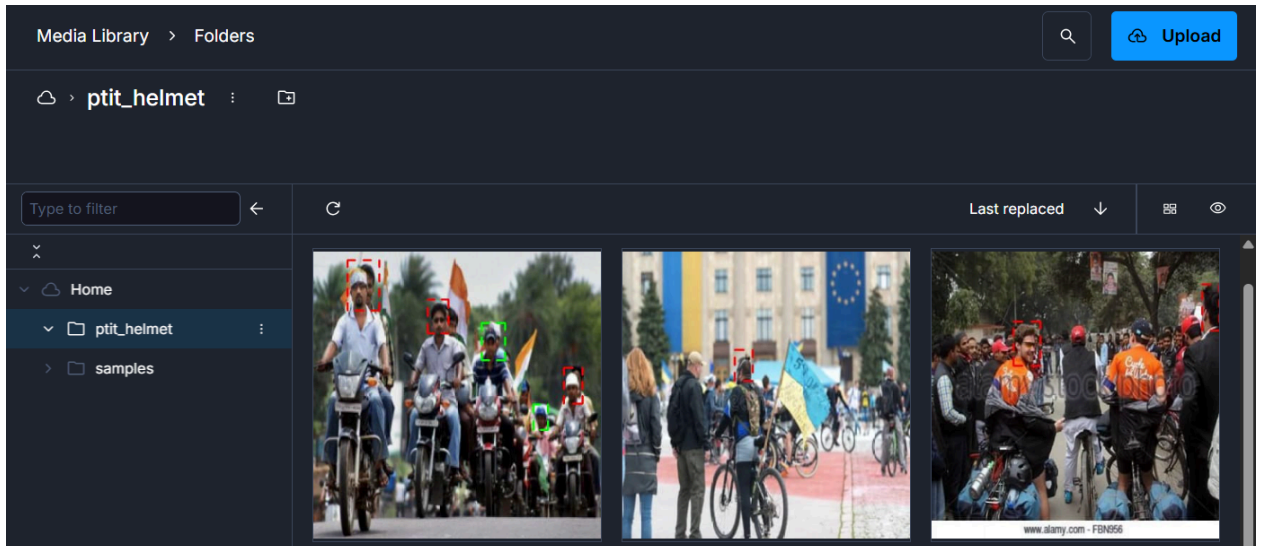
- Lưu ảnh vi phạm lên cloud

- Trả về URL ảnh
- Giảm tải bộ nhớ server

Quy trình:

Detection => Encode image => Upload Cloudinary => Lưu URL vào MongoDB

Ảnh minh họa:



2.2.3. Cơ chế chống spam (Cooldown)

Một vấn đề phát sinh trong quá trình xử lý video là:

- Một đối tượng vi phạm có thể bị detect liên tục trong nhiều frame

Điều này dẫn đến:

- Lưu trùng dữ liệu
- Tăng tải database
- Tốn tài nguyên cloud

Giải pháp:

- Xây dựng cơ chế Cooldown (ALERT_COOLDOWN)
- Chỉ cho phép lưu một vi phạm trong một khoảng thời gian (ví dụ: 10 giây)

Kết quả:

- Giảm đáng kể số lượng bản ghi trùng lặp
- Tối ưu tài nguyên hệ thống

2.3. Tùy biến hiển thị bằng OpenCV Custom Drawing

Để ảnh bằng chứng chuyên nghiệp hơn, em đã thay thế hàm vẽ mặc định của YOLO bằng hàm vẽ custom sử dụng OpenCV

Cách thực hiện:

- Tự vẽ bounding box bằng OpenCV
- Phân biệt màu sắc theo lớp:
- Without Helmet: màu đỏ (cảnh báo)
- With Helmet: màu xanh
- Áp dụng style nét đứt

Mục đích:

- Tăng tính rõ ràng khi quan sát
- Giảm che khuất đối tượng
- Đồng nhất giữa livestream và ảnh lưu trữ

2.4. Tái cấu trúc API nhận diện ảnh

Sau khi thiết lập kiến trúc phân lớp, em đã tiến hành refactor lại toàn bộ API nhận diện ảnh tĩnh (/helmet/predict) được xây dựng từ tuần trước.

- Tách biệt logic: Chuyển toàn bộ mã nguồn vẽ bounding box và lưu kết quả sang cách module phù hợp, không để tất cả trong helmet_service
- Đồng bộ hóa công cụ vẽ: Áp dụng hàm vẽ custom cho cả API ảnh tĩnh. Điều này giúp hình ảnh trả về cho người dùng và hình ảnh lưu trữ trên Cloudinary luôn đồng nhất về mặt hiển thị.
- Sử dụng Background Tasks: Tương tự như luồng video, API ảnh tĩnh cũng được tích hợp tác vụ chạy ngầm để upload bằng chứng lên mây, giúp giảm thời gian chờ của người dùng xuống mức tối thiểu.

2.5. Xây dựng API truy xuất lịch sử vi phạm

Để hoàn thiện hệ thống quản lý, em đã phát triển thêm module /violations chuyên trách việc truy xuất dữ liệu từ MongoDB Atlas.

- Tính năng phân trang (Pagination): Thay vì lấy toàn bộ dữ liệu (có thể lên tới hàng ngàn bản ghi gây lag hệ thống), API hỗ trợ các tham số page và limit. Kỹ thuật này giúp hệ thống hoạt động mượt mà và tiết kiệm băng thông.

- Sắp xếp thời gian thực: Dữ liệu được truy vấn và sắp xếp theo thứ tự thời gian mới nhất lên đầu (timestamp: -1), giúp người quản lý dễ dàng theo dõi các sự kiện vừa mới xảy ra.
- Chuẩn hóa dữ liệu trả về: API trả về đầy đủ thông tin bao gồm: số lượng vi phạm, thời gian, danh sách tọa độ bounding box và đặc biệt là đường dẫn ảnh (URL) từ Cloudinary để hiển thị trực tiếp trên giao diện Dashboard.

3. Chuẩn hóa kiến trúc và bảo mật hệ thống

Tuần này đánh dấu bước chuyển mình về mặt kỹ thuật phần mềm (Software Engineering) cho dự án.

3.1. Áp dụng kiến trúc phân lớp (Layered Architecture)

Project được tổ chức theo các package chuyên biệt, bổ sung một số package mới:

- app/exceptions: Quản lý lỗi tập trung.
- app/utils: Chứa các hàm hỗ trợ (vẽ hình, xử lý path).

Việc tách biệt này giúp code trở nên Clean, dễ bảo trì và mở rộng khi thêm các module khác sau này.

3.2. Quản lý cấu hình bằng file .env

Toàn bộ thông tin nhạy cảm được tách ra khỏi source code và lưu trong file .env, bao gồm:

- MongoDB URI
- Cloudinary API Key
- Các tham số cấu hình hệ thống

Sử dụng:

- pydantic_settings.BaseSettings để load config
- Truy cập thông qua object setting

Lợi ích:

- Bảo mật thông tin
- Dễ thay đổi môi trường (dev/production)
- Tránh hard-code trong codebase

3.3. Xử lý tác vụ ngầm (Background Tasks)

Trong hệ thống, một số tác vụ có độ trễ cao như:

- Upload ảnh lên Cloudinary
- Ghi dữ liệu vào MongoDB

Nếu xử lý trực tiếp trong request sẽ gây:

- Tăng thời gian phản hồi
- Làm giạt luồng video

Giải pháp:

- Sử dụng BackgroundTasks của FastAPI
- Đẩy các tác vụ này chạy ngầm sau khi trả response

Luồng xử lý:

Detect => Return response => Background: upload + save DB

Kết quả:

- API phản hồi nhanh hơn
- Luồng livestream ổn định
- Tránh block hệ thống

3.4. Hệ thống xử lý lỗi tập trung (Global Exception Handling)

Để đảm bảo tính nhất quán của dữ liệu trả về và nâng cao khả năng bảo trì, em đã xây dựng một cơ chế quản lý ngoại lệ tập trung cho toàn bộ hệ thống Backend.

- Xây dựng phân cấp lỗi (Error Hierarchy): Em thiết lập một lớp lỗi gốc `AppError` kế thừa từ `Exception` của Python. Từ đó, các lỗi đặc thù như `ImageDecodingError` (lỗi giải mã ảnh) hay `InvalidFileFormat` (sai định dạng tệp) được định nghĩa thành các lớp con riêng biệt.
- Cơ chế Handler tập trung: Thay vì sử dụng quá nhiều khối `try-except` gây loãng code tại tầng Router, em sử dụng tính năng `exception_handler` của FastAPI để đăng ký một bộ xử lý lỗi toàn cục.
- Tính nhất quán của dữ liệu trả về: Mọi lỗi xảy ra trong quá trình vận hành, từ lỗi logic nghiệp vụ đến lỗi hệ thống không xác định, đều được tự động chuyển đổi sang định dạng JSON đồng nhất: `{"detail": "ErrorMessage"}` kèm theo mã trạng thái HTTP (Status Code) phù hợp (400, 404, 500...).

Lợi ích: Cách tiếp cận này giúp tăng Service tập trung hoàn toàn vào logic xử lý ảnh và AI, đồng thời giúp phía Frontend dễ dàng bắt lỗi và hiển thị thông báo chính xác cho người dùng cuối.

4. Kết quả đạt được trong tuần 5

Sau khi hoàn thành các công việc trong tuần thứ năm, các kết quả chính đạt được bao gồm:

- Xây dựng thành công hệ thống giám sát video thời gian thực đạt chuẩn 30 FPS.
- Thiết lập hoàn chỉnh kết nối tới cơ sở dữ liệu MongoDB Atlas trên Cloud.
- Tích hợp thành công Cloudinary, giải quyết triệt để bài toán lưu trữ hình ảnh nặng.
- Chuẩn hóa toàn bộ hệ thống API: Refactor thành công các API cũ theo kiến trúc mới, giúp code sạch, dễ đọc và dễ mở rộng.
- Hoàn thiện luồng dữ liệu khép kín: Từ khâu Nhận diện (YOLO) => Xử lý hiển thị (OpenCV) => Lưu trữ (Cloudinary/MongoDB) => Truy xuất (History API).
- Triển khai hệ thống xử lý ngoại lệ (Exception Handling) chuyên nghiệp, trả về mã lỗi JSON nhất quán.

5. Khó khăn gặp phải và cách khắc phục

Trong quá trình thực hiện tuần thứ năm của dự án, một số khó khăn đã phát sinh liên quan đến việc duy trì sự ổn định của luồng video trực tiếp, tối ưu hóa quy trình lưu trữ trên nền tảng Hybrid-Cloud (MongoDB Atlas và Cloudinary), cũng như tái cấu trúc mã nguồn theo các chuẩn mực kỹ thuật phần mềm hiện đại. Các vấn đề này chủ yếu xuất phát từ yêu cầu khắt khe về việc đảm bảo hiệu năng xử lý thời gian thực, tính nhất quán của dữ liệu khi vận hành liên tục và nỗ lực xây dựng một kiến trúc hệ thống bền bỉ, có khả năng bảo trì và mở rộng linh hoạt trong tương lai.

5.1. Lỗi SSL Handshake khi kết nối MongoDB Atlas

Trong quá trình tích hợp cơ sở dữ liệu MongoDB Atlas từ môi trường Windows, hệ thống phát sinh lỗi:

TLSV1_ALERT_INTERNAL_ERROR

Nguyên nhân chủ yếu xuất phát từ sự không tương thích hoặc xung đột giữa các chứng chỉ bảo mật (SSL/TLS) trên môi trường local và cấu hình của MongoDB Atlas.

Vấn đề này khiến ứng dụng không thể thiết lập kết nối tới database, làm gián đoạn quá trình phát triển và kiểm thử hệ thống.

Để khắc phục, em đã:

- Tạm thời cấu hình MongoDB Atlas cho phép truy cập từ tất cả địa chỉ IP (0.0.0.0/0)
- Kiểm tra lại cấu hình URI kết nối và thư viện driver

Giải pháp này giúp:

- Đảm bảo kết nối thông suốt trong môi trường phát triển
- Cho phép tiếp tục triển khai các chức năng liên quan đến database

Tuy nhiên, đây chỉ là giải pháp tạm thời. Trong môi trường production, cần giới hạn IP và cấu hình bảo mật chặt chẽ hơn để đảm bảo an toàn hệ thống.

5.2. Vấn đề trễ khi lưu trữ đám mây

Khi tích hợp chức năng lưu ảnh vi phạm lên Cloudinary, hệ thống gặp phải tình trạng giật, lag trong quá trình xử lý video realtime.

Nguyên nhân:

- Quá trình upload ảnh lên cloud có độ trễ tương đối lớn
- Nếu thực hiện trực tiếp trong luồng xử lý chính sẽ làm block pipeline xử lý video

Hậu quả:

- Giảm FPS
- Ảnh hưởng đến trải nghiệm livestream

Để giải quyết vấn đề này, em đã:

- Sử dụng BackgroundTasks của FastAPI
- Tách toàn bộ logic upload ảnh và lưu database sang luồng xử lý ngầm
- Chỉ truyền dữ liệu cần thiết sang background task

5.3. Khó khăn trong việc xử lý lỗi

Trong giai đoạn đầu xây dựng API, việc xử lý lỗi chưa được tổ chức thống nhất, dẫn đến:

Các thông báo lỗi bị phân tán trong nhiều file

- Khó kiểm soát và debug
- Response trả về không đồng nhất

Điều này gây khó khăn cho cả backend và frontend khi cần xử lý lỗi.

Để khắc phục, em đã:

- Tái cấu trúc module app/exceptions
- Tách riêng các thành phần:
 - base.py: định nghĩa lớp lỗi chung
 - helmet.py: lỗi riêng cho module helmet
 - handlers.py: xử lý lỗi tập trung (global handler)

Cách tiếp cận mới:

Raise Error => Global Handler => JSON Response

Kết quả:

- Router trở nên đơn giản, chỉ còn xử lý request/response
- Lỗi được quản lý tập trung và nhất quán
- Dễ mở rộng và bảo trì hệ thống

5.4 Khó khăn trong việc tùy biến hiển thị thị giác máy tính

Trong quá trình xây dựng hệ thống giám sát, việc hiển thị bounding box đóng vai trò quan trọng đối với trải nghiệm người dùng.

Ban đầu, hệ thống sử dụng phương pháp vẽ mặc định của YOLO. Tuy nhiên, cách hiển thị này có một số hạn chế:

- Bounding box dạng nét liền, chiếm nhiều diện tích
- Có thể che khuất đối tượng trong khung hình
- Khó quan sát trong môi trường nhiều đối tượng

Để cải thiện, em đã tiến hành tùy biến việc hiển thị bằng OpenCV.

Khó khăn gặp phải:

- OpenCV không hỗ trợ trực tiếp vẽ đường nét đứt
- Cần tự xây dựng hàm vẽ từ các đoạn thẳng nhỏ

Giải pháp:

- Tự implement hàm vẽ bounding box dạng nét đứt
- Điều chỉnh độ dài và khoảng cách giữa các đoạn

Kết quả:

- Giảm mức độ che khuất đối tượng
- Tăng khả năng quan sát trong video
- Giao diện hiển thị trực quan và chuyên nghiệp hơn

6. Kế hoạch thực hiện trong tuần tiếp theo

Sau khi đã xây dựng được mô hình và bước đầu triển khai hệ thống API trong tuần thứ năm, tuần tiếp theo của dự án sẽ tập trung vào việc hoàn thiện hệ thống theo hướng ứng dụng thực tế. Các công việc được đề ra nhằm nâng cao hiệu năng, mở rộng chức năng và cải thiện trải nghiệm người dùng.

6.1. Xây dựng giao diện Web Dashboard

Sau khi hệ thống backend đã được hoàn thiện ở mức có thể xử lý dữ liệu và lưu trữ, bước tiếp theo là xây dựng giao diện người dùng nhằm trực quan hóa và khai thác dữ liệu một cách hiệu quả.

Trong giai đoạn này, em dự kiến phát triển một Web Dashboard sử dụng các công nghệ frontend hiện đại như ReactJS hoặc HTML/CSS/JavaScript thuần.

Các chức năng chính của dashboard bao gồm:

- Hiển thị video giám sát theo thời gian thực:
 - Tích hợp khung hình livestream từ camera thông qua API streaming đã xây dựng ở backend. Người dùng có thể quan sát trực tiếp quá trình phát hiện vi phạm.
- Hiển thị danh sách vi phạm:
 - Các sự kiện vi phạm sẽ được trình bày dưới dạng các thẻ (card), bao gồm:
 - Ảnh bằng chứng (lấy từ Cloudinary)
 - Thời gian xảy ra vi phạm
 - Thông tin đối tượng phát hiện
- Tương tác với hệ thống:

- Cho phép người dùng dễ dàng theo dõi, kiểm tra và quản lý các sự kiện vi phạm thông qua giao diện trực quan.

Việc xây dựng dashboard giúp:

- Chuyển hệ thống từ dạng backend sang ứng dụng hoàn chỉnh
- Cải thiện trải nghiệm người dùng
- Tăng khả năng ứng dụng trong thực tế

6.2. Phát triển tính năng lọc và thống kê dữ liệu

Bên cạnh việc hiển thị dữ liệu, hệ thống cần cung cấp khả năng phân tích để hỗ trợ người dùng theo dõi và đánh giá tình hình vi phạm.

Trong giai đoạn này, em sẽ phát triển các chức năng:

- Bộ lọc dữ liệu vi phạm:
 - Lọc theo ngày, tháng hoặc khoảng thời gian
 - Cho phép truy vấn nhanh các sự kiện theo nhu cầu
- Thống kê và trực quan hóa dữ liệu:
 - Tính toán số lượng vi phạm theo thời gian
 - Biểu diễn dữ liệu dưới dạng biểu đồ (bar chart, line chart)
 - Hiển thị xu hướng thay đổi của các hành vi vi phạm

Các chức năng này giúp:

- Người dùng dễ dàng theo dõi tình hình tổng thể
- Hỗ trợ phân tích và đưa ra quyết định
- Nâng cao giá trị thực tiễn của hệ thống